

Guía Didáctica: Patrón Factory - Creación Inteligente de Objetos

Índice

1. [¿Qué es el Patrón Factory?](#)
2. [¿Por qué es necesario? El Problema del new Directo](#)
3. [Tipos de Factory Pattern](#)
4. [Factory Method vs Abstract Factory](#)
5. [Implementación en TypeScript](#)
6. [Caso de Estudio: Concesionaria de Vehículos](#)
7. [Factory en Arquitectura Hexagonal](#)
8. [Ventajas y Desventajas](#)
9. [Anti-patrones: Qué NO Hacer](#)
10. [Ejercicios Prácticos](#)
11. [Checklist de Implementación](#)

1. ¿Qué es el Patrón Factory?

DEFINICIÓN

PATRÓN FACTORY

=====

Es un patrón de diseño
creacional
que proporciona una
interfaz para
crear objetos en una
superclase,
pero permite a las subclases
alterar
el tipo de objetos que se
crearán.



ANALOGÍA: FÁBRICA DE AUTOS



Fábrica
(Interfaz de creación)



Auto Sedán
(Producto concreto)



Auto SUV
(Producto concreto)



Auto Camioneta
(Producto concreto)

El Patrón en Una Frase

"No llames a `new` directamente; usa un método que decida qué crear por ti."

PATRÓN FACTORY EN POCAS PALABRAS

✖ SIN FACTORY:

```
const auto = new AutoSedan('Toyota', 'Corolla');  
const camioneta = new Camioneta('Ford', 'Ranger');
```

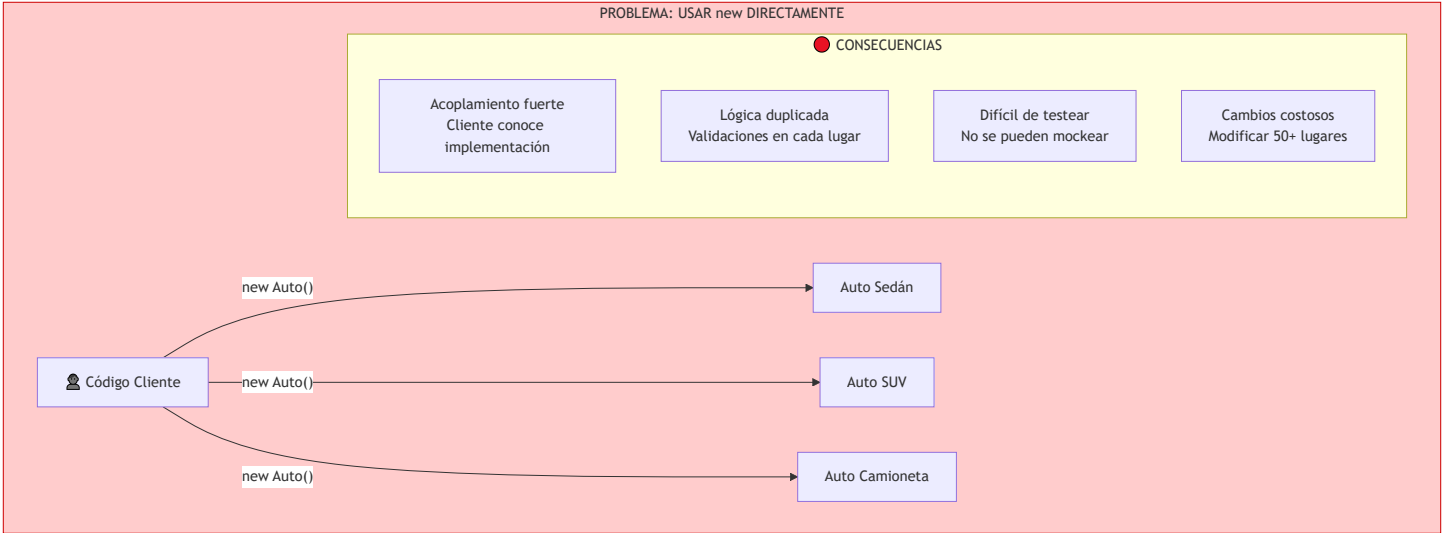
✔ CON FACTORY:

```
const auto = FabricaAutos.crear('sedan', 'Toyota', 'Corolla');  
const camioneta = FabricaAutos.crear('camioneta', 'Ford', 'Ranger');
```

🎯 EL CLIENTE NO SABE CÓMO SE CONSTRUYE EL OBJETO

2. ¿Por qué es necesario? El Problema del `new` Directo

El Problema Visual



Ejemplo Concreto del Problema

// ❌ CÓDIGO SIN FACTORY (EL PROBLEMA)

```
class AutoSedan {
    constructor(
        private _marca: string,
        private _modelo: string,
        private _precio: number
    ) {
        // Validación DUPLICADA en cada clase
        if (!_marca || _marca.trim().length < 2) {
            throw new Error("Marca inválida");
        }
        if (_precio < 0) {
            throw new Error("Precio no puede ser negativo");
        }
    }
}
```

```
class AutoSUV {
    constructor(
        private _marca: string,
        private _modelo: string,
        private _precio: number
    ) {
        // ❌ La MISMA validación duplicada
        if (!_marca || _marca.trim().length < 2) {
            throw new Error("Marca inválida");
        }
        if (_precio < 0) {
            throw new Error("Precio no puede ser negativo");
        }
    }
}
```

```
class AutoCamioneta {
    constructor(
        private _marca: string,
        private _modelo: string,
        private _precio: number
    ) {
        // ❌ La MISMA validación duplicada NUEVAMENTE
```

```

    if (!_marca || _marca.trim().length < 2) {
        throw new Error("Marca inválida");
    }
    if (_precio < 0) {
        throw new Error("Precio no puede ser negativo");
    }
}
}

// ❌ EL CLIENTE TIENE QUE SABER QUÉ TIPO CREAR
function crearAuto(tipo: string, marca: string, modelo: string, precio: number) {
    // ❌ Lógica de selección en el cliente
    if (tipo === 'sedan') {
        return new AutoSedan(marca, modelo, precio);
    } else if (tipo === 'suv') {
        return new AutoSUV(marca, modelo, precio);
    } else if (tipo === 'camioneta') {
        return new AutoCamioneta(marca, modelo, precio);
    }
    throw new Error("Tipo no soportado");
}

// ❌ SI AGREGAMOS UN NUEVO TIPO, HAY QUE MODIFICAR ESTE CÓDIGO

```

Problemas Identificados

Problema	Manifestación	Impacto
Validaciones duplicadas	Misma lógica en 3+ clases	Bugs inconsistentes
Acoplamiento fuerte	Cliente conoce todas las clases	Difícil de cambiar
Lógica de selección dispersa	Cada cliente hace su propio switch	Código repetido
Open/Closed violado	Para agregar nuevo tipo, modificas código existente	Violación de SOLID
Difícil de testear	No puedes mockear fácilmente	Pruebas frágiles

3. Tipos de Factory Pattern

TIPOS DE FACTORY PATTERN

Abstract Factory

Fábrica de fábricas
Crea familias de objetos
relacionados

- ✓ Coherencia entre productos
- ✗ Complejidad alta

Factory Method (Patrón)

Una interfaz con método
create()
Cada subclase implementa
su creación

- ✓ Abierto a extensión
- ✓ Sigue Open/Closed

Factory Method (Simple Factory)

Una clase con un método
estático
que decide qué objeto
crear

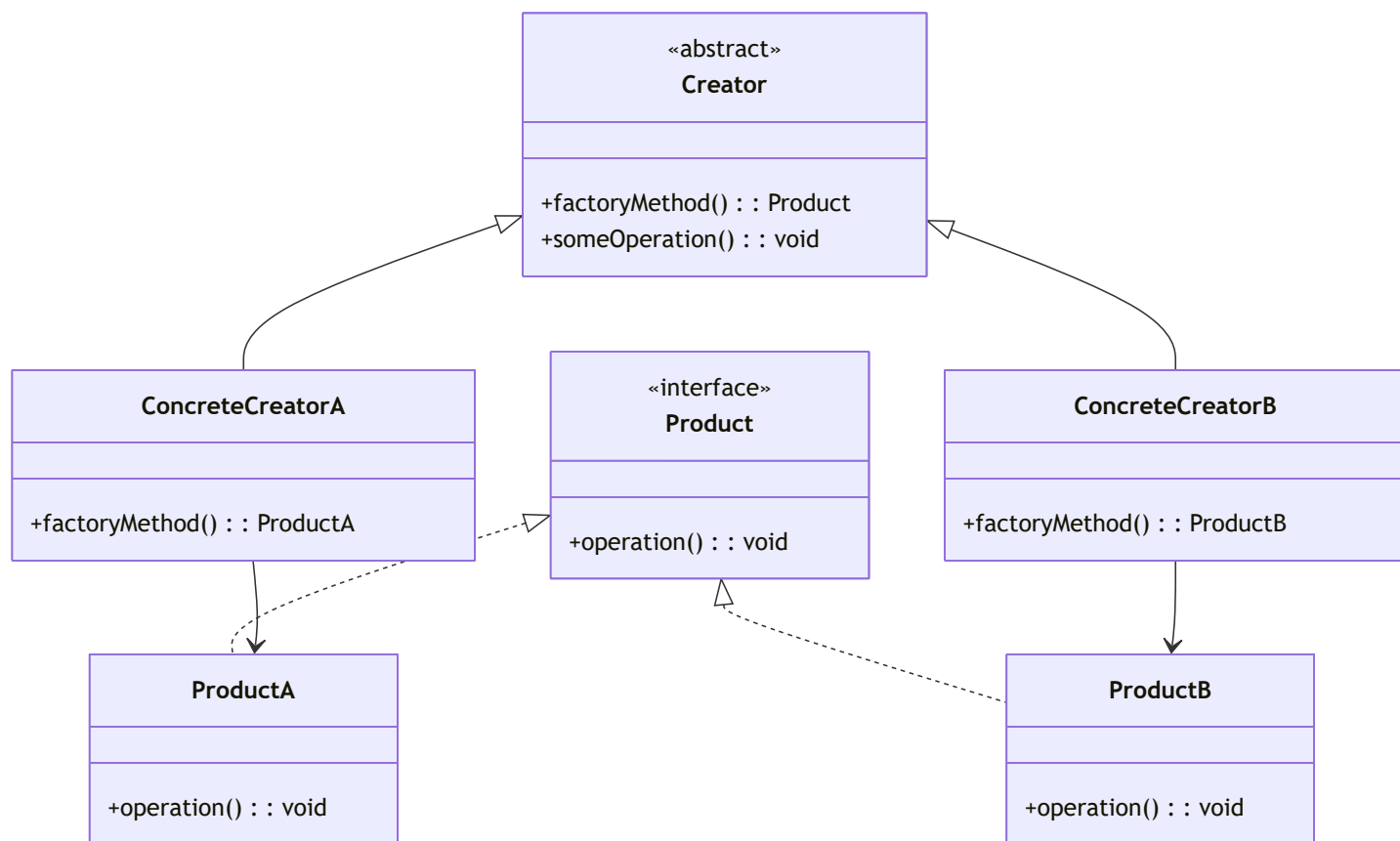
- ✓ Simple de implementar
- ✗ Viola Open/Closed

Comparativa de Tipos

Tipo	Complejidad	Flexibilidad	Uso típico
Simple Factory	Baja	Baja	Cuando solo hay un tipo de producto
Factory Method	Media	Alta	Cuando hay variantes de un mismo producto
Abstract Factory	Alta	Muy alta	Cuando hay familias de productos relacionados

4. Factory Method vs Abstract Factory

Factory Method



Ejemplo en Código:


```
// Producto
interface Vehiculo {
    conducir(): void;
}

// Productos concretos
class Sedan implements Vehiculo {
    conducir(): void {
        console.log("Conduciendo un sedán cómodo");
    }
}

class SUV implements Vehiculo {
    conducir(): void {
        console.log("Conduciendo una SUV robusta");
    }
}

// Creador Abstracto
abstract class FabricaVehiculos {
    // Factory Method
    abstract crearVehiculo(): Vehiculo;

    // Operación que usa el factory method
    entregarVehiculo(): void {
        const vehiculo = this.crearVehiculo();
        vehiculo.conducir();
    }
}

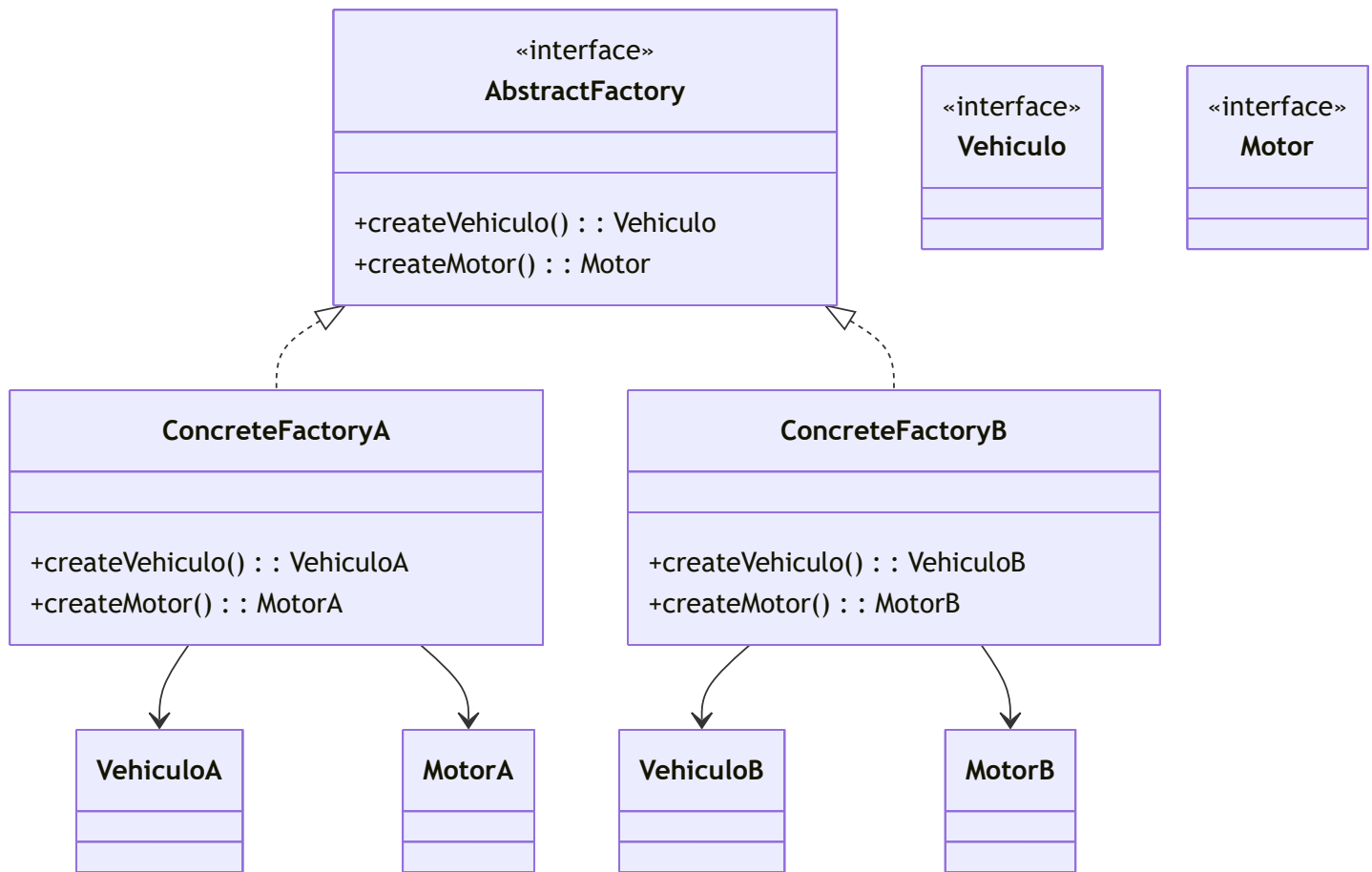
// Creadores Concretos
class FabricaSedan extends FabricaVehiculos {
    crearVehiculo(): Vehiculo {
        return new Sedan();
    }
}

class FabricaSUV extends FabricaVehiculos {
    crearVehiculo(): Vehiculo {
        return new SUV();
    }
}
```

```
// Uso
const fabrica1 = new FabricaSedan();
fabrica1.entregarVehiculo(); // Conduciendo un sedán cómodo

const fabrica2 = new FabricaSUV();
fabrica2.entregarVehiculo(); // Conduciendo una SUV robusta
```

Abstract Factory



Ejemplo en Código:

```
// Productos
interface Vehiculo {
    conducir(): void;
}

interface Motor {
    encender(): void;
}

// Productos Concretos (Familia A - Sedan)
class Sedan implements Vehiculo {
    conducir(): void {
        console.log("Sedán: Conducción suave");
    }
}

class MotorSedan implements Motor {
    encender(): void {
        console.log("Motor Sedán: Rrrrrr...");
    }
}

// Productos Concretos (Familia B - SUV)
class SUV implements Vehiculo {
    conducir(): void {
        console.log("SUV: Conducción robusta");
    }
}

class MotorSUV implements Motor {
    encender(): void {
        console.log("Motor SUV: RRRRRRRR!!");
    }
}

// Abstract Factory
interface FabricaAutomotriz {
    crearVehiculo(): Vehiculo;
    crearMotor(): Motor;
}

// Factories Concretas
class FabricaSedan implements FabricaAutomotriz {
```

```

    crearVehiculo(): Vehiculo {
        return new Sedan();
    }
    crearMotor(): Motor {
        return new MotorSedan();
    }
}

class FabricaSUV implements FabricaAutomotriz {
    crearVehiculo(): Vehiculo {
        return new SUV();
    }
    crearMotor(): Motor {
        return new MotorSUV();
    }
}

// Uso
const fabrica: FabricaAutomotriz = new FabricaSedan();
const vehiculo = fabrica.crearVehiculo();
const motor = fabrica.crearMotor();

vehiculo.conducir(); // Sedán: Conducción suave
motor.encender();    // Motor Sedán: Rrrrrr...

```

5. Implementación en TypeScript

Simple Factory (La más común en tu proyecto)

```
// 📁 domain/factories/employee.factory.ts

import { Employee, EmployeeData } from '../entities/employee.entity.js';
import { v4 as uuidv4 } from 'uuid';

export class EmployeeFactory {
  /**
   * Crea un nuevo empleado (NUEVO registro)
   * - Genera ID único
   * - Establece fechas de creación
   * - Valida datos de entrada
   */
  static create(
    cedula: string,
    nombres: string,
    salario: number
  ): Employee {
    // Validaciones de dominio (en el factory)
    if (!cedula || cedula.trim().length < 10) {
      throw new Error('La cédula debe tener al menos 10 caracteres');
    }

    if (!nombres || nombres.trim().length < 2) {
      throw new Error('Los nombres deben tener al menos 2 caracteres');
    }

    if (salario < 0) {
      throw new Error('El salario no puede ser negativo');
    }

    const id = `emp-${Date.now()}-${Math.random().toString(36).substr(2, 9)}`;
    const now = new Date();

    // Crear la entidad (constructor privado)
    return Employee.fromData({
      id,
      cedula: cedula.trim(),
      nombres: nombres.trim(),
```

```

        salario,
        createdAt: now,
        updatedAt: now
    });
}

/**
 * Reconstruye un empleado desde datos persistidos
 */
static fromData(data: EmployeeData): Employee {
    // Validar que los datos sean consistentes
    if (!data.id) {
        throw new Error('ID requerido para reconstruir empleado');
    }

    return Employee.fromData(data);
}

/**
 * Clona un empleado (para crear copias)
 */
static clone(employee: Employee): Employee {
    const data = employee.toJSON();
    return EmployeeFactory.create(
        data.cedula,
        data.nombres,
        data.salario
    );
}
}

```

Factory con Dependencias (Inyección)

```
// 📁 application/factories/employee.factory.ts

import { Employee } from '../../domain/entities/employee.entity.js';
import { EmployeeRepository } from '../../domain/ports/employee.repository.js';
import { v4 as uuidv4 } from 'uuid';

interface CreateEmployeeParams {
  cedula: string;
  nombres: string;
  salario: number;
  departmentId?: string;
}

export class EmployeeFactory {
  constructor(
    private readonly repository: EmployeeRepository,
    private readonly idGenerator: () => string = uuidv4
  ) {}

  async createEmployee(params: CreateEmployeeParams): Promise<Employee> {
    // 1. Validar que no exista ya
    const exists = await this.repository.existsByCedula(params.cedula);
    if (exists) {
      throw new Error(`Ya existe un empleado con cédula ${params.cedula}`);
    }

    // 2. Validar reglas de negocio
    this.validate(params);

    // 3. Crear la entidad
    const id = this.idGenerator();
    const now = new Date();

    const employee = Employee.fromData({
      id,
      cedula: params.cedula.trim(),
      nombres: params.nombres.trim(),
      salario: params.salario,
      departmentId: params.departmentId,
      createdAt: now,
      updatedAt: now
    });
  }
}
```

```

    });

    return employee;
}

private validate(params: CreateEmployeeParams): void {
    if (!params.cedula || params.cedula.length < 10) {
        throw new Error('Cédula inválida');
    }
    if (!params.nombres || params.nombres.length < 2) {
        throw new Error('Nombres inválidos');
    }
    if (params.salario < 0) {
        throw new Error('Salario no puede ser negativo');
    }
    if (params.salario > 1000000) {
        throw new Error('Salario excede el límite permitido');
    }
}

}

// Uso en el Service
export class EmployeeService {
    constructor(
        private readonly repository: EmployeeRepository,
        private readonly factory: EmployeeFactory
    ) {}

    async createEmployee(request: CreateEmployeeRequest): Promise<Employee> {
        // El factory se encarga de la creación
        const employee = await this.factory.createEmployee(request);

        // El service se encarga de la persistencia
        await this.repository.save(employee);

        return employee;
    }
}

```


6. Caso de Estudio: Concesionaria de Vehículos

Contexto

La concesionaria necesita crear diferentes tipos de vehículos con diferentes configuraciones.

Implementación con Factory

```
// =====  
// DOMAIN - Entidades  
// =====  
  
interface Vehiculo {  
    id: string;  
    marca: string;  
    modelo: string;  
    precio: number;  
    tipo: TipoVehiculo;  
    conducir(): void;  
}  
  
enum TipoVehiculo {  
    SEDAN = 'SEDAN',  
    SUV = 'SUV',  
    HATCHBACK = 'HATCHBACK',  
    PICKUP = 'PICKUP'  
}  
  
// =====  
// DOMAIN - Implementaciones concretas  
// =====  
  
class VehiculoSedan implements Vehiculo {  
    constructor(  
        public readonly id: string,  
        public readonly marca: string,  
        public readonly modelo: string,  
        public readonly precio: number,  
        public readonly tipo: TipoVehiculo = TipoVehiculo.SEDAN  
    ) {}  
  
    conducir(): void {  
        console.log(`🚗 Conduciendo ${this.marca} ${this.modelo} (Sedán) - Cómodo y eficiente`)  
    }  
}  
  
class VehiculoSUV implements Vehiculo {  
    constructor(  
        public readonly id: string,
```

```

        public readonly marca: string,
        public readonly modelo: string,
        public readonly precio: number,
        public readonly tipo: TipoVehiculo = TipoVehiculo.SUV,
        public readonly traccion: string = '4x4'
    ) {}

    conducir(): void {
        console.log(`🚗 Conduciendo ${this.marca} ${this.modelo} (SUV) - Robusto y todoterreno`
    )
}

// =====
// APPLICATION - Factory Interface
// =====

interface VehiculoFactory {
    crearVehiculo(marca: string, modelo: string, precio: number): Vehiculo;
}

// =====
// APPLICATION - Factories Concrete
// =====

class SedanFactory implements VehiculoFactory {
    crearVehiculo(marca: string, modelo: string, precio: number): Vehiculo {
        return new VehiculoSedan(
            this.generarId(),
            marca,
            modelo,
            precio
        );
    }

    private generarId(): string {
        return `SED-${Date.now()}-${Math.random().toString(36).substr(2, 6)}`;
    }
}

class SUVFactory implements VehiculoFactory {
    crearVehiculo(marca: string, modelo: string, precio: number): Vehiculo {
        return new VehiculoSUV(
            this.generarId(),

```

```

        marca,
        modelo,
        precio
    );
}

private generarId(): string {
    return `SUV-${Date.now()}-${Math.random().toString(36).substr(2, 6)}`;
}
}

```

```

// =====
// APPLICATION - Factory Dinámica (Simple Factory)
// =====

```

```

class FabricaVehiculos {
    static crear(
        tipo: TipoVehiculo,
        marca: string,
        modelo: string,
        precio: number
    ): Vehiculo {
        const fabricas: Record<TipoVehiculo, VehiculoFactory> = {
            [TipoVehiculo.SEDAN]: new SedanFactory(),
            [TipoVehiculo.SUV]: new SUVFactory(),
            [TipoVehiculo.HATCHBACK]: new HatchbackFactory(),
            [TipoVehiculo.PICKUP]: new PickupFactory(),
        };

        const factory = fabricas[tipo];
        if (!factory) {
            throw new Error(`Tipo de vehículo no soportado: ${tipo}`);
        }

        return factory.crearVehiculo(marca, modelo, precio);
    }
}

```

```

// =====
// APPLICATION - Abstract Factory (Familia completa)
// =====

```

```

interface FabricaAutomotrizCompleta {

```

```

    crearVehiculo(): Vehiculo;
    crearMotor(): Motor;
    crearTransmision(): Transmission;
}

```

```

class FabricaSedanCompleta implements FabricaAutomotrizCompleta {
    crearVehiculo(): Vehiculo {
        return new VehiculoSedan('SED-001', 'Toyota', 'Corolla', 22000);
    }
    crearMotor(): Motor {
        return new MotorSedan();
    }
    crearTransmision(): Transmission {
        return new TransmissionAutomatica();
    }
}

```

```

// =====
// INFRASTRUCTURE - Uso en Controller
// =====

```

```

class VehiculoController {
    private readonly vehiculoService: VehiculoService;

    crearVehiculo = async (req: Request, res: Response): Promise<void> => {
        try {
            const { tipo, marca, modelo, precio } = req.body;

            //  Usando Factory para crear el vehículo
            const vehiculo = FabricaVehiculos.crear(
                tipo as TipoVehiculo,
                marca,
                modelo,
                precio
            );

            const guardado = await this.vehiculoService.save(vehiculo);

            res.status(201).json({
                message: 'Vehículo creado exitosamente',
                data: guardado
            });
        }
    }
}

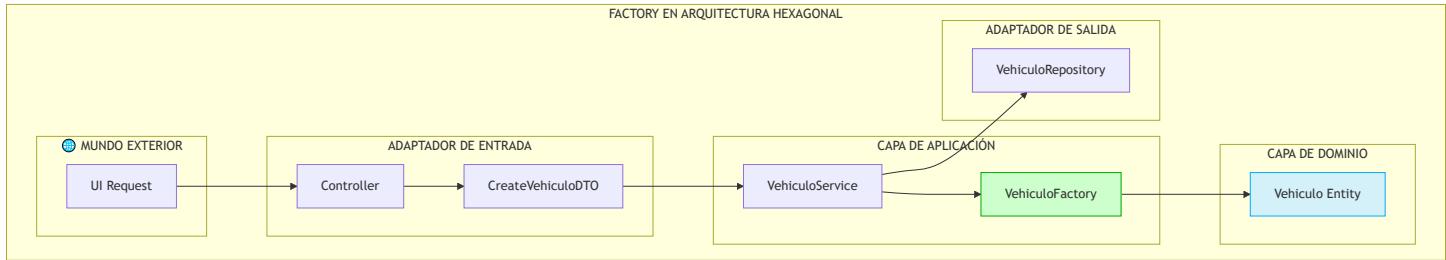
```

```

    } catch (error) {
        res.status(400).json({ error: error.message });
    }
};
}

```

7. Factory en Arquitectura Hexagonal

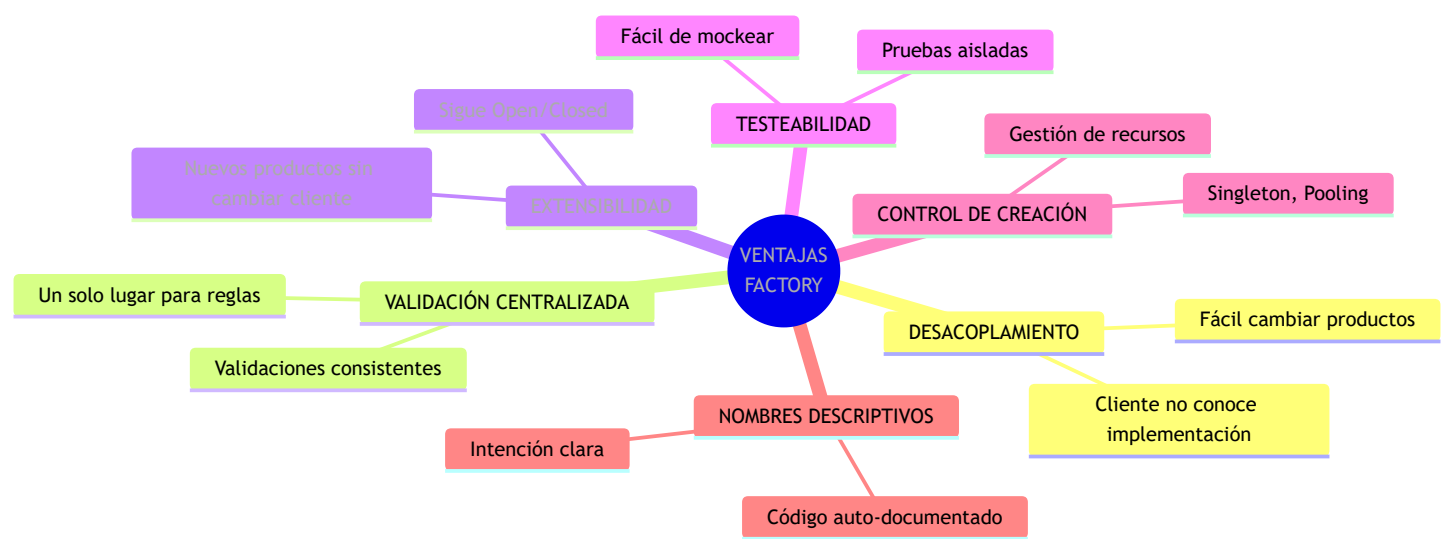


Ubicación del Factory en Hexagonal

Capa	¿Dónde poner Factory?	Razón
Dominio	✅ Recomendado	Valida reglas de negocio, crea entidades
Aplicación	⚠️ Opcional	Cuando depende de repositorios para validar
Infraestructura	❌ No	Demasiado específico

8. Ventajas y Desventajas

Ventajas



Desventajas

Desventaja	Impacto	Mitigación
Complejidad adicional	Más código para mantener	Solo usar cuando sea necesario
Indirección	Más difícil de seguir	Documentar bien
Over-engineering	Si no se necesita	Evaluar si realmente es necesario

9. Anti-patrones: Qué NO Hacer

ANTI-PATRONES A EVITAR

✗ Factory con LÓGICA DE NEGOCIO

El factory solo debe crear,
no hacer reglas complejas

✗ Factory GIGANTE

Más de 10 tipos en un solo
factory

✗ Factory que CONOCE IMPLEMENTACIONES

El factory no debe saber
detalles internos de los
productos

✗ Factory con EFECTOS SECUNDARIOS

No debe guardar, notificar,
etc.

✗ Factory sin VALIDACIONES

Debe validar antes de crear

Ejemplo de Anti-patrón

```
// ❌ ANTI-PATRÓN: Factory que hace DEMASIADO

class BadEmployeeFactory {
    static create(data: any): Employee {
        // ❌ 1. Efecto secundario (guardar en BD)
        this.repository.save(data);

        // ❌ 2. Lógica de negocio compleja
        if (data.salario > 5000) {
            data.bonus = data.salario * 0.1;
            // ❌ 3. Notificaciones
            this.emailService.send('...');
        }

        // ❌ 4. Lógica de selección compleja
        if (data.tipo === 'gerente') {
            return new Gerente(data);
        } else if (data.tipo === 'vendedor') {
            return new Vendedor(data);
        }

        // ❌ 5. Validaciones que deberían estar en la entidad
        if (data.cedula.length < 10) {
            throw new Error('...');
        }

        // ... y más código
    }
}
```

Versión Correcta

//  FACTORY CORRECTO: Solo crea

```
class GoodEmployeeFactory {
    static create(cedula: string, nombres: string, salario: number): Employee {
        // 1. Validaciones BÁSICAS (formato de datos)
        if (!cedula || cedula.trim().length < 10) {
            throw new Error('Cédula inválida');
        }

        // 2. CREAR la entidad (constructor privado)
        return Employee.fromData({
            id: this.generateId(),
            cedula: cedula.trim(),
            nombres: nombres.trim(),
            salario,
            createdAt: new Date(),
            updatedAt: new Date()
        });
    }

    private static generateId(): string {
        return `emp-${Date.now()}-${Math.random().toString(36).substr(2, 9)}`;
    }
}
```

//  La lógica de negocio y efectos secundarios van en el SERVICE

```
class EmployeeService {
    async createEmployee(data: any): Promise<Employee> {
        // 1. Validaciones de negocio
        if (await this.repository.exists(data.cedula)) {
            throw new Error('Ya existe');
        }

        // 2. Crear entidad (USANDO FACTORY)
        const employee = GoodEmployeeFactory.create(
            data.cedula,
            data.nombres,
            data.salario
        );

        // 3. Persistir
```

```

    await this.repository.save(employee);

    // 4. Notificar
    await this.notificationService.send('Empleado creado');

    return employee;
  }
}

```

10. Ejercicios Prácticos

Ejercicio 1: Factory para Planes de Financiamiento

```

// Crea un factory que cree diferentes tipos de PlanesFinanciamiento
// basado en el perfil del cliente

```

```

enum PerfilCliente {
  BASICO = 'BASICO',
  PREMIUM = 'PREMIUM',
  VIP = 'VIP'
}

```

```

interface PlanFinanciamiento {
  tasaInteres: number;
  plazoMaximo: number;
  engancheMinimo: number;
}

```

```

// TODO: Implementar PlanFactory

```

```

class PlanFactory {
  // ... tu código aquí
}

```

```

// Ejemplo de uso esperado:
// const plan = PlanFactory.crear(PerfilCliente.PREMIUM);
// plan.tasaInteres === 0.08;
// plan.plazoMaximo === 60;

```

Ejercicio 2: Factory con Dependencias

```
// Crea un factory que use un repositorio para validar
// y un generador de IDs

interface IdGenerator {
    generate(): string;
}

class EmployeeFactory {
    constructor(
        private readonly repository: EmployeeRepository,
        private readonly idGenerator: IdGenerator
    ) {}

    // TODO: Implementar método create
    // Debe validar que no exista el empleado antes de crearlo
}
```

Ejercicio 3: Abstract Factory para Reportes

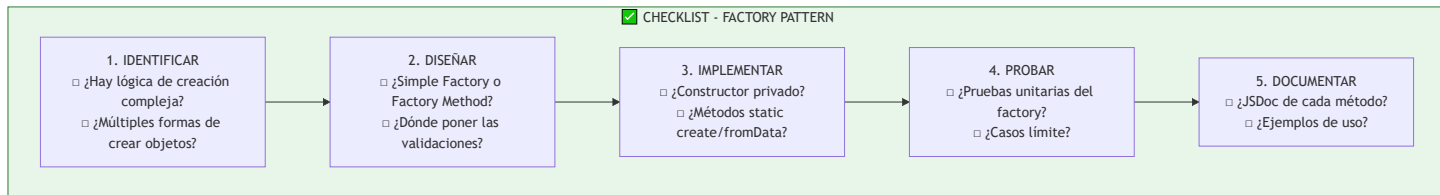
```
// Crea una fábrica abstracta para generar diferentes tipos de reportes

interface Reporte {
    generar(): string;
}

interface ReporteFactory {
    crearReporteVentas(): Reporte;
    crearReporteComisiones(): Reporte;
    crearReporteInventario(): Reporte;
}

// TODO: Implementar factorías para diferentes formatos (PDF, Excel, HTML)
```

11. Checklist de Implementación



Preguntas Clave para Decidir

Pregunta	Respuesta	Acción
¿El objeto tiene constructor complejo?	Sí	Usar Factory
¿Hay validaciones al crear?	Sí	Usar Factory
¿Múltiples formas de crear?	Sí	Usar Factory Method
¿Se crean familias de objetos?	Sí	Usar Abstract Factory
¿Es un objeto simple (POJO)?	Sí	No necesitas Factory

Resumen Final



